

PRACTICAL WORK 1 DOCUMENTATION

C++ Project

1) Introduction

-The implementation of the directed graph is done using three internal dictionaries(`std::unordered_map`):

- inbound: stores the vertex as a key and a vector of vertices that have edges toward it
- outbound: stores the vertex as a key and a vector of vertices that it points to
- cost: stores a pair of vertices(source, target) as key and an integer as the edge cost

2) Main Classes

a) Class GraphClass

-implements the internal structure of the graph

-members: vertices:

- stores the number of vertices
- inbound and outbound: maps for incoming and outgoing edges
- cost: map `std::pair<int,int>` for storing edge costs

-important methods

- `getCostDict()` – returns a reference to the cost dictionary
- `getVertices()` – returns the number of vertices
- `getEdges()` – returns the number of edges
- `getInbound(int vertex) / getOutbound(int vertex)` – return vectors of inbound/outbound neighbours
- `addEdge(int from, int to, int edgeCost) / removeEdge(int from, int to)` – modify edges
- `changeCost(int x, int y, int newCost)` – modify the cost of an existing edge
- `getVertexList()` – return a vector with all vertices

-methods using `at()` will throw `std::out_of_range` if a vertex does not exist

b) Class GraphRepo

-handles file operations and random graph generation

-members:

- filename: stores the name of the loaded/saved file
- graph: pointer to the GraphClass object

-methods:

- loadFile(filename) – reads a graph from a text file and returns a GraphClass*
- saveFile(fileName) – writes the current graph to a text file
- createRandomGraph(n, m) – generates a graph with n vertices and m edges with random costs(1-100)
- getGraph() – return a pointer to the current graph

-random edges are generated using std::random_device and std::mt19937

-duplicate edges are avoided

c) Class GraphService

-provides operations on the graph safely with preconditions checked

-methods:

- addEdge(x, y, cost) / removeEdge(x, y) – adds or removes an edge with error checking
- addVertex(v) / removeVertex(v) – adds or removes a vertex with error checking
- copyGraph() – returns a deep copy of the graph(GraphService*)
- getInbound(vertex) / getOutbound(vertex) / getCost(x, y) – return adjacency list or edge costs
- changeCost(x, y, c) – modifies the cost of an existing edge
- getVertices() / getEdges() / getVertexList() – return graph information

-exceptions(std::runtime_error, std::out_of_range) are used to handle invalid operations

-all operations are efficient: checking or retrieving an edge, accessing inbound/outbound edges, getting endpoints or costs, and computing degrees are all $O(1)$ amortized.

d) Class Menu

-provides a console interface to test all operations

-uses GraphRepo and GraphService for functionality

-displays a menu and allows the user to:

- load/save graphs, generate random graphs
- add/remove vertices and edges
- copy the graph
- display vertices, edges, in/out degrees and costs

-input validation is included: if the user enters an invalid vertex or edge, an exception message is printed and the user is asked again